Local Nix without Root

Rohit Goswami · 2020-09-07 · workflow · projects · hpc · nix · tools

> *Monkeying around with `nix` for HPC systems which have no root access and NFS filesystems.*

Background

Nix is not well known for being friendly to users without root access. This is typically made worse by the "exotic" filesystem attributes common to HPC networks (this also plagues hermes). An earlier post details how and why `proot` failed. The short pitch is simply:

Figure 1: Does your HPC look like this?

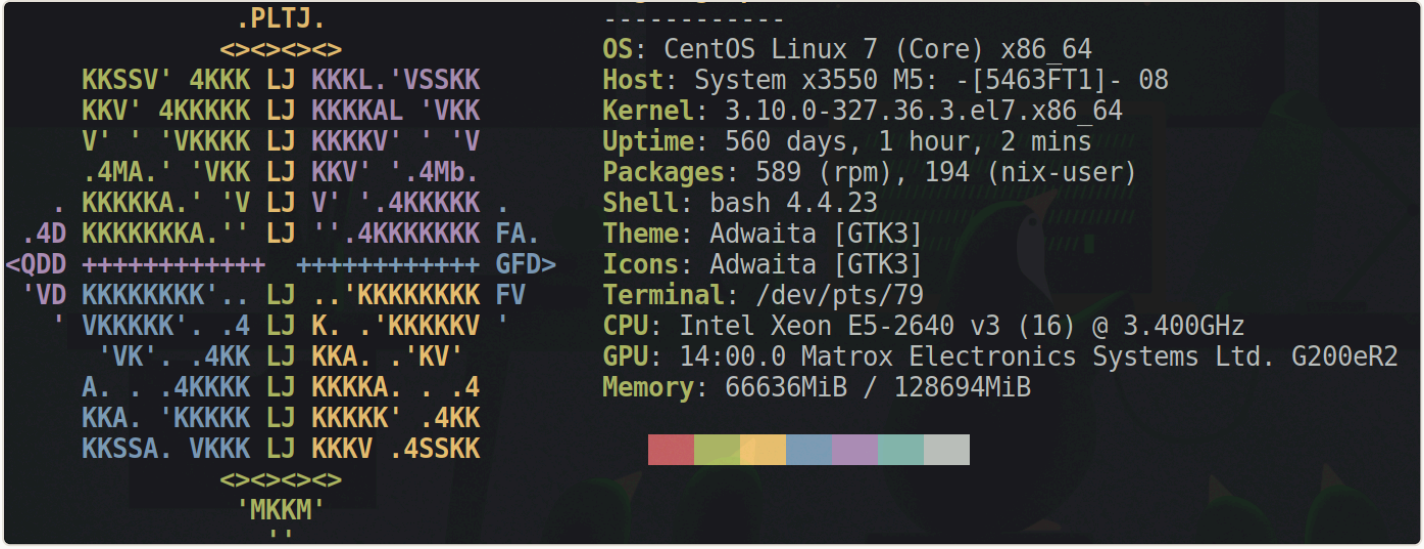

Figure 2: It really is an HPC

If your HPC doesn't look that swanky and you'd like it to, then read on. Note that there are all the obvious benefits of `nix` as well, but this is a more eye-catchy pitch.

Setup

> *The basic concept is to install `nix` from source, with appropriate patches, and then mess around with paths until it is ready and willing to work with stores which are not `/nix` [^fn:1]*

This concept is strongly influenced by the work described in this repo. The premise is similar to my earlier post on HPC Dotfiles. For the purposes of this post, we will assume that all the packages in the previous post exist. `lmod` is not required, feel free to use an alternative path management system, or even just `$HOME/.local` but if `lmod` is present, it is highly recommended [^fn:2]. We **will need** the following:

Pinned set of nixpkgs

We would like to be able to modify a lot of paths, which is normally a bad practice, but then we don't normally rebuild all packages either. Grab a copy of the `nixpkgs` by following the instructions below. Now is also the time to fork the repo if you'd like to keep track of your changes.

```
mkdir -p $HOME/Git/Github
cd $HOME/Git/Github
git clone https://github.com/NixOS/nixpkgs
```

dotgit

We use the older, bash version of the excellent dotgit since `python` is not always present in HPC environments.

```
git clone https://github.com/kobus-v-schoor/dotgit/
mkdir -p $HOME/.local/bin
cp dotgit/old/bin/bash_completion dotgit/old/bin/dotgit dotgit/old/bin/dotgit_headers
dotgit/old/bin/fish_completion.fish $HOME/.local/bin/ -r
```

lmod packages

If you do not or cannot use modulefiles as described in the earlier post, inspect the module-files being loaded and set paths accordingly.

```
cd $HOME/Git/Github
git clone https://github.com/HaoZeke/hzHPC_lmod
cd hzHPC_lmod
$HOME/.local/bin/dotgit restore hzhpc
```

Now we can start by obtaining the `nix` sources.

```
myprefix=$HOME/.hpc/nix/nix-boot
nixdir=$HOME/.nix
nix_version=2.3.7
ml load gcc/9.2.0 flex bison
ml load boost
ml load editline
ml load brotli/1.0.1
ml load libseccomp/2.4.4
ml load bdwgc/8.0.4
ml load bzip2/1.0.8
ml load sqlite
ml load patch xz
wget http://nixos.org/releases/nix/nix-${nix_version}/nix-${nix_version}.tar.bz2
tar xfv nix-2.3.7.tar.bz2
cd nix-2.3.7
```

Before actually configuring and installing from source, we need some patches.

Patches

I suggest carefully typing out the patches, though leave a comment if you want a repo with these changes (if you must star something in the meantime, star this).

- Start with this patch

```
wget https://github.com/NixOS/nix/commit/8d3cb66d22f348341d7afa626acfa53b40584fdd.patch
git apply 8d3cb66d22f348341d7afa626acfa53b40584fdd.patch
```

- Also this one

Remove the following `ifdef` stuff from `src/libutil/compression.cc`, leaving only the contents of the `else` statement.

```
#ifdef HAVE_LZMA_MT
            lzma_mt mt_options = {};
            mt_options.flags = 0;
            mt_options.timeout = 300; // Using the same setting as the xz cmd line
            mt_options.preset = LZMA_PRESET_DEFAULT;
            mt_options.filters = NULL;
            mt_options.check = LZMA_CHECK_CRC64;
            mt_options.threads = lzma_cputhreads();
            mt_options.block_size = 0;
            if (mt_options.threads == 0)
                mt_options.threads = 1;
            // FIXME: maybe use lzma_stream_encoder_mt_memusage() to control the
            // number of threads.
            ret = lzma_stream_encoder_mt(&strm, &mt_options);
            done = true;
#else
            printMsg(lvlError, "warning: parallel XZ compression requested but not supported, falling back to
single-threaded compression");
#endif
```

If there is trouble with the bzip2 library, set `$HOME/.hpc/bzip2/1.0.8/include/bzlib.h` in `src/libutil/compression.cc`, but expand `$HOME`.

Finally, you will need edit `nixpkgs`.

```
# vim pkgs/os-specific/linux/busybox/default.nix
  debianName = "busybox_1.30.1-6";
  debianTarball = fetchzip {
    url = "http://deb.debian.org/debian/pool/main/b/busybox/${debianName}.debian.tar.xz";
    sha256 = "05n6mxc8n4zsli4dijrr2x5c9ggwi223i5za4n0xwhgd4lkhqymw";
  };
```

User Build

We can now complete the build.

```
ml load openssl curl
./configure  --enable-gc --prefix=$myprefix --with-store-dir=$nixdir/store --localstatedir=$nixdir/var --with-
boost=$BOOST_ROOT --disable-seccomp-sandboxing --disable-doc-gen --with-sandbox-shell=/usr/bin/sh CPPFLAGS="-
I$HOME/.hpc/bzip2/1.0.8/include" LDFLAGS="-L$HOME/.hpc/bzip2/1.0.8/lib -Wl,-R$HOME/.hpc/bzip2/1.0.8/lib"
make -j $(nproc)
make install
ml load nix/user # Hooray!
ml unload openssl curl
```

Now we still need to set a profile. Inspect `.hpc/nix/nix-boot/etc/profile.d/nix.sh` and check the value of `NIX_PROFILES`

```
chmod +x .hpc/nix/nix-boot/etc/profile.d/nix.sh
./.hpc/nix/nix-boot/etc/profile.d/nix.sh
# OR, and this is better
nix-env --switch-profile .nix/var/nix/profiles/default
mkdir -p  ~/.nix/var/nix/profiles
```

The reason why we need to switch profiles is because by default `nix-env --switch-profile` will use `/nix/var/nix/profiles/per-user/$USER/profile` in a multi-user setup, and it is better to keep this where we have control as well.

We also need to kill the sandbox for now, as also seen in the AUR package (and here).

```
# ~/.config/nix/nix.conf
sandbox = false
substituters = https://cache.nixos.org https://all-hies.cachix.org
trusted-public-keys = cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQFGspcDShjY= all-hies.cachix.org-
1:JjrzAOEUsD9ZMt8fdFbzo3jNAyEWlPAwdVuHw4RD43k=
```

Now we can test this before moving forward:

```
nix-channel --update
nix-shell -p hello
```

Rebuilding Natively

The astute reader will have noticed that we glibly monkeyed around with the `nix` source in the previous section, but all will be made well since we can rebuild to use `nix` with itself. Do **replace the variable with the corresponding path**:

```
storeDir = "$HOME/.nix/store";
stateDir = "$HOME/.nix/var";
confDif = "$HOME/.nix/etc";
```

Essentially, the `$HOME/.config/nixpkgs/config.nix` should look like (incorporating both the patches and also the full directory we will be using):

```
{
  packageOverrides = pkgs:
    with pkgs; {
      autogen = autogen.overrideAttrs (oldAttrs: {
        postInstall = ''
          mkdir -p $dev/bin
          mv $bin/bin/autoopts-config $dev/bin
          for f in $lib/lib/autogen/tpl-config.tlib $out/share/autogen/tpl-config.tlib; do
            sed -e "s|$dev/include|/no-such-autogen-include-path|" -i $f
            sed -e "s|$bin/bin|/no-such-autogen-bin-path|" -i $f
            sed -e "s|$lib/lib|/no-such-autogen-lib-path|" -i $f
          done
          # remove /tmp/** from RPATHs
          for f in "$bin"/bin/*; do
            local nrp="$(patchelf --print-rpath "$f" | sed -E 's@(:|^)/tmp/[^:]*:@\1@g')"
            patchelf --set-rpath "$nrp" "$f"
          done
        '' + stdenv.lib.optionalString (!stdenv.hostPlatform.isDarwin) ''
          # remove /build/** from RPATHs
          for f in "$bin"/bin/*; do
            local nrp="$(patchelf --print-rpath "$f" | sed -E 's@(:|^)/build/[^:]*:@\1@g')"
            patchelf --set-rpath "$nrp" "$f"
          done
        '';
      });
      nix = nix.overrideAttrs (oldAttrs: {
        storeDir = "/users/home/jdoe/.nix/store";
        stateDir = "/users/home/jdoe/.nix/var";
        confDif = "/users/home/jdoe/.nix/etc";
        doCheck = false;
        doInstallCheck = false;
        prePatch = ''
          substituteInPlace src/libstore/local-store.cc \
            --replace '(eaName == "security.selinux")' \
                      '(eaName == "security.selinux" || eaName == "system.nfs4_acl")'
          substituteInPlace src/libstore/gc.cc \
            --replace 'auto mapLines =' \
                      'continue; auto mapLines ='
          substituteInPlace src/libstore/sqlite.cc \
            --replace 'SQLITE_OPEN_READWRITE | SQLITE_OPEN_CREATE, 0) != SQLITE_OK)' \
                      'SQLITE_OPEN_READWRITE | SQLITE_OPEN_CREATE, "unix-dotfile") != SQLITE_OK)'
        '';
      });
    };
}
```

We can “speed up” our build by disabling all tests. Go to the copy of `nixpkgs` and run:

```
find pkgs  -type f -name 'default.nix' | xargs sed -i 's/doCheck = true/doCheck = false/'
```

```
mkdir -p $HOME/.nix/var/nix/profiles/
nix-env -i nix -f $HOME/Git/Github/nixpkgs -j$(nproc) --keep-going --show-trace -v --cores 4 2>&1 | tee nix-
no-root.log
ml load nix/bootstrapped
```

This will still take a couple of hours at least. Around 3-4 hours. Try to set this up on a lazy weekend to evade sysadmins.

If `curl 429` rate limits are encountered for `musl` sources, the solution is to replace the source (put the following in a file, say `no429.patch`):

```
diff --git a/pkgs/os-specific/linux/musl/default.nix b/pkgs/os-specific/linux/musl/default.nix
index ae175a3..1a6f6c7 100644
--- a/pkgs/os-specific/linux/musl/default.nix
+++ b/pkgs/os-specific/linux/musl/default.nix
@@ -4,12 +4,12 @@
 }:
 let
   cdefs_h = fetchurl {
-    url = "http://git.alpinelinux.org/cgit/aports/plain/main/libc-dev/sys-cdefs.h";
+    url = "https://raw.githubusercontent.com/akadata/aports/master/main/libc-dev/sys-cdefs.h";
     sha256 = "16l3dqnfq0f20rzbkhc38v74nqcsh9n3f343bpczqq8b1rz6vfrh";
   };
   queue_h = fetchurl {
-    url = "http://git.alpinelinux.org/cgit/aports/plain/main/libc-dev/sys-queue.h";
-    sha256 = "12qm82id7zys92a1qh2l1qf2wqgq6jr4qlbjmqyfffz3s3nhfd61";
+    url = "https://raw.githubusercontent.com/akadata/aports/master/main/libc-dev/sys-queue.h";
+    sha256 = "049pd547ckrsky72s18a649mz660yph14wdrlw9gnbk903skdnz4";
   };
   tree_h = fetchurl {
     url = "http://git.alpinelinux.org/cgit/aports/plain/main/libc-dev/sys-tree.h";
```

This can be applied with `git apply`.

Usage

We have finally obtained a bootstrapped `nix` which is bound to our set of `nixpkgs`. To ensure its use:

```
ml use $HOME/Modulefiles
ml purge
ml load nix/bootstrapped
ml save
```

Flakes and DevShells

Newer versions of `nix` depend on `mdbook` which is meant for generating the documentation. Unfortunately, the `cargo256` hashes are path dependent. A quick fix is to remove the dependency on `mdbook` and disable documentation generation with the following ugly patch:

```
diff --git a/pkgs/tools/package-management/nix/default.nix b/pkgs/tools/package-management/nix/default.nix
index 7eda5ae..91bf1b8 100644
--- a/pkgs/tools/package-management/nix/default.nix
+++ b/pkgs/tools/package-management/nix/default.nix
@@ -14,7 +14,7 @@ common =
   , pkg-config, boehmgc, libsodium, brotli, boost, editline, nlohmann_json
   , autoreconfHook, autoconf-archive, bison, flex
   , jq, libarchive, libcpuid
-  , lowdown, mdbook
+  , lowdown
   # Used by tests
   , gtest
   , busybox-sandbox-shell
@@ -36,7 +36,7 @@ common =

       VERSION_SUFFIX = suffix;

-      outputs = [ "out" "dev" "man" "doc" ];
+      outputs = [ "out" "dev" ];

       nativeBuildInputs =
         [ pkg-config ]
@@ -45,7 +45,6 @@ common =
           [ autoreconfHook
             autoconf-archive
             bison flex
-            (lib.getBin lowdown) mdbook
             jq
            ];

@@ -119,8 +118,8 @@ common =
         [ "--with-store-dir=${storeDir}"
           "--localstatedir=${stateDir}"
           "--sysconfdir=${confDir}"
-          "--disable-init-state"
           "--enable-gc"
+          "--disable-doc-gen"
         ]
         ++ lib.optionals stdenv.isLinux [
           "--with-sandbox-shell=${sh}/bin/busybox"
@@ -136,7 +135,8 @@ common =

       installFlags = [ "sysconfdir=$(out)/etc" ];

-      doInstallCheck = true; # not cross
+      doInstallCheck = false; # not cross
+      doCheck = false;

       # socket path becomes too long otherwise
       preInstallCheck = lib.optionalString stdenv.isDarwin ''
@@ -160,7 +160,7 @@ common =
         license = lib.licenses.lgpl2Plus;
         maintainers = [ lib.maintainers.eelco ];
         platforms = lib.platforms.unix;
-        outputsToInstall = [ "out" "man" ];
+        outputsToInstall = [ "out" ];
       };

       passthru = {
```

We also need to update our `config.nix`:

```
nixUnstable = nixUnstable.overrideAttrs (oldAttrs: {
    storeDir = "/users/home/rog32/.nix/store";
    stateDir = "/users/home/rog32/.nix/var";
    confDif = "/users/home/rog32/.nix/etc";
    doCheck = false;
    doInstallCheck = false;
    prePatch = ''
      substituteInPlace src/libstore/local-store.cc \
        --replace '(eaName == "security.selinux")' \
                  '(eaName == "security.selinux" || eaName == "system.nfs4_acl")'
      substituteInPlace src/libstore/gc.cc \
        --replace 'auto mapLines =' \
                  'continue; auto mapLines ='
     '';
   });
```

Now we can finally get to the installation of a newer version. I prefer to live life on the edge:

```
nix-env -iA nixUnstable -f $HOME/Git/Github/nixpkgs -j$(nproc) --keep-going --show-trace --cores 4 2>&1 | tee
nix-install-base.log
```

We are now able activate flakes and other features like `nix shell` (note the space!).

```
# ~/.config/nix/nix.conf
experimental-features = nix-command flakes
```

Bonus: Fixing Documentation

In order to get the original derivation working, we need to essentially modify the `cargo256` hashes. Thankfully the `nix` build log is rather verbose.

```
installing
error: hash mismatch in fixed-output derivation '/users/home/jdoen/.nix/sto$
e/n71nkimlbazmq1vpyyavqcxzg9c86brs-mdbook-0.4.7-vendor.tar.gz.drv':
         specified: sha256-2kBJcImytsSd7Q0kj1bsP/NXxyy2Pr8gHb8iNf6h3/4=
            got:    sha256-4bYLrmyI7cPUes6DYREiIB9gDze0KO2jMP/jPzvWbwQ=
error: 1 dependencies of derivation '/users/home/jdoen/.nix/store/wr31pgva8a
zn9jvvpa4bshykv80xf5qi-mdbook-0.4.7.drv' failed to build
error: 1 dependencies of derivation '/users/home/jdoen/.nix/store/y8pkc0hhgz
rvxgrj7c00mmsy50plya6p-nix-2.4pre20210326_dd77f71.drv' failed to build
```

We need to modify `pkgs/tools/text/mdbook/default.nix` to update the hash; and then:

```
nix-env -iA nixUnstable -f $HOME/Git/Github/nixpkgs -j$(nproc) --keep-going --show-trace --cores 4 2>&1 | tee
nix-install-base.log
ml load nix/bootstrapped
nix-shell --help # Works
nix-shell -p hello # Also works
```

Channels

We would like to move away from having to constantly pass our cloned set of packages.

```
nix-channel --add https://nixos.org/channels/nixpkgs-unstable nixpkgs
nix-channel --update
```

Basic Packages

Now we can get some basic stuff too.

```
nix-env -i tmux zsh lsof pv git -f $HOME/Git/Github/nixpkgs -j$(nproc) --keep-going --show-trace --cores 4
2>&1 | tee nix-install-base.log
```

Ruby Caveats

> *No longer relevant as of April 2020*

While installing packages which depend on `ruby`, there will be permission errors inside the build folder. These can be "fixed" by setting very permissive controls on the **build-directory** in question. **Do not** set permissions directly on the `.nix/store/$HASH` folder, as doing so will make `nix` reject the build artifact.

```
# neovim depends on ruby
nix-env -i neovim -v -f $HOME/Git/Github/nixpkgs
```

A more elegant way to fix permissions involves a slightly more convoluted approach. We can note where the build is occurring (e.g. `/tmp`) and run a `watch` command to fix permissions.

```
watch -n1 -x chmod 777 -R /tmp/nix-build-ruby-2.6.6.drv-0/source/lib/
```

Naturally this must be run in a separate window.

Dotfiles

Feel free to set up `dotfiles` (mine, perhaps) to profit even further. We will consider the process of obtaining my set below. Minimally, we will want to obtain `tmux` and `zsh`.

```
nix-env -i tmux zsh -v -f $HOME/Git/Github/nixpkgs
```

Now we can set the `dotfiles` up.

```
git clone https://github.com/HaoZeke/Dotfiles
cd Dotfiles
$HOME/.local/bin/dotgit restore hzhpc
```

The final installation configures `neovim` and `tmux`.

```
zsh
# Should install things with zinit
tmux
# CTRL+b --> SHIFT+I to install
nvim
```

Misc NFS

For issues concerning NFS lock files, consider simply moving the problematic file and let things sort themselves out. Consider:

```
nix-build
# something about a .nfs lockfile in some .nix/$HASH-pkg/.nfs0234234
mv .nix/$HASH-pkg/ .diePKGs/
nix-build # profit
```

The right way to deal with this is of course:

```
nix-build
lsof +D .nix/$HASH-pkg/.nfs0234234
kill $whatever_blocks
nix-build # profit
```

Conclusions

Though this is slow and seems like an inefficient use of cluster resources, the benefits of reproducible environments typically outweighs the cost. Also it is much more pleasant to have a proper package manager which can work with Dotfiles.
